# Project Announcement

| Component | Deadline | Team/Individual | Points | Symbol |
|---|---|---|---|---|
| Midterm written report | Dec. 2, 2026 | Team | 5 pts | $M$ |
| Final oral report | Week 15 | Individual | 25 pts | $O$ |
| In-class spotlight presentation | Dec. 30, 2026 | Team | Bonus | $B$ |
| Final written report | Jan. 13, 2027 | Individual | 10 pts | $W$ |
| Source-code submission | Jan. 13, 2027 | Team | Coefficient | $k$ |

The total project score is 40 points, plus bonus points.

$$S = M + k(O + W) + B.$$

Here, $O$ denotes the final individual oral report score, assessed based on both the demonstrated results and the student's technical explanation. $W$ denotes the final written report score after any deductions.

The code-verification coefficient $k$ ranges from 0 to 1 and is initially set to 1. It may be reduced if source-code verification reveals material discrepancies between the submitted implementation and the claimed functionality or individual contribution.

## 1 Midterm Written Report

Please submit a two-page report summarizing your project progress using the SIGGRAPH template. Include the following:

- Your chosen project topic
- Your project goal and the technical components you plan to implement
- A detailed schedule for the entire project
- The technical components you have completed, the methods you have used, and any available visual results
- Your plan for the remaining technical components
- The external tools you have used or plan to use

## 2 Final Individual Oral Report

Each student is required to give an individual presentation of approximately 5 minutes to a TA during Week 15, outside regular class hours. The presentation will be followed by a 3 minute Q&A with the TAs.

The oral report will be evaluated in two aspects:

- **Visual presentation:** Prepare a short slide deck or other presentation materials, and present clear visual results and a live demo that show the quality of the project and your specific contributions.
- **Technical discussion:** Explain your methods and implementation, then answer the TA's questions about your work and its technical details. Your score will reflect the clarity and correctness of your explanation and your understanding of the implementation.

Based on the final individual oral reports, the TAs will select approximately 20 teams to present their projects in class during Week 16.

# 3 In-Class Spotlight Presentation

Approximately 20 teams selected by the TAs will give spotlight presentations in class during Week 16 (Dec 30, 2026), showcasing their projects with visual results and live demos.

All enrolled students will have the opportunity to vote for their favorite projects. The most popular projects will receive up to 3 bonus points, and all teams that give an in-class spotlight presentation will receive 2 bonus points.

# 4 Final Individual Written Report

Each student must submit a report describing their own contributions using the SIGGRAPH template. The report must be at least three pages long and include an introduction, methods, results, discussion, a personal contribution statement, and references. Each student must write their report independently. **Sharing written content, even within the same team, is not permitted.**

In addition, you must submit the source code for the project (a GitHub link is recommended). Whether working individually or in a team, proper commit management is advised to help TAs evaluate your workload. In the final written report, clearly indicate which parts of the code you contributed to.

# 5 Projects

In the following sections, the red parts represent the core requirements for this topic. **You must fully implement these components**; failure to do so will result in your project being considered incomplete. It is acceptable if you choose not to pursue the optional parts, but skipping the entire project is not allowed.

For the grading of oral report, there are two possible scenarios:

- **Individual:** You must complete the red parts, which will automatically earn you 10pts. Afterward, you may optionally select any combination of the **black parts** to earn up to an additional 15pts.
- **Team of two:** Your team must complete the red parts, which will automatically earn each individual 15pts. Then, each individual may optionally select a **different combination** of the **black parts** to earn up to an additional 10pts (totally 20pts as a team). This means the red parts are completed collaboratively, but only one team member can receive points for each of the black parts.

Final written report is not scored separately for each project part. Instead,

$$W = 10 - \min\left(10, \sum_i d_i\right),$$

where $d_i$ is the deduction for part $i$ due to an incomplete explanation or a mismatch between the report and the implementation.

Overall, we encourage everyone to complete the project as a team. Since the high difficulty of some technical points, we believe that completing 15pts individually is comparable in difficulty to completing 20pts as a team, so the total workload is similar. However, if you choose to work as a team, you are more likely to achieve higher completion and presentation quality, which can help you earn better scores and possibly bonuses during the spotlight presentation.

If you wish to implement any advanced techniques in your project, please contact the TA **before the Oral Report** to discuss feasibility and grading criteria.

## 5.1 Topic 1: Image Rendering

In this section, you are required to implement a renderer, which can be either hardware-based (GPU) or software-based (CPU). The goal is to produce a static image. Note that a path-tracing renderer is required, as rasterization-based renderers will not be accepted. The functionalities you need to implement are as follows:

- **Base:** Implement a path tracing algorithm that correctly handles **diffuse** and **specular** materials. (basic)
- **Scene creation:** Build a custom scene with aesthetic considerations, using geometry that you create from scratch or find online (ensure the source is credited). (basic, tidiness and attractiveness 1pt)
- **Acceleration structure:** Implement an acceleration structure such as BVH (Bounding Volume Hierarchy). This is not required for hardware-based renderers, as the acceleration structure is built-in in that case. (basic, Surface Area Heuristic or another advanced algorithm 2pts)
- **Material:** Create a (non-trivial) custom material. Options include:
  - Transmissive material (basic)
  - Principled BSDF (2pts)
  - Multi-layer material (2pts)
  - Rendering of fur, hair, skin, etc. (2pts)
- **Texture:** Create your own (non-trivial) texture with proper texture mapping. Options include:
  - Color texture (basic)
  - Normal map, height map, attribute map, or any functional texture mapping (1pt for each, up to 2pts)
  - Implement an adaptive mipmap algorithm (2pts)
- **Importance Sampling:** Use more advanced sampling algorithms for path tracing. (Importance sampling with Russian Roulette, multiple importance sampling 2pts)
- **Volumetric Rendering:** Options include:
  - Subsurface scattering (2pts)
  - Homogeneous volume rendering (1pt)
  - Inhomogeneous volume rendering (1pt)
  - Channel-independent subsurface scattering (1pt)
  - Volumetric emission (1pt)
  - Volumetric alpha shadow (2pts)
- **Special Visual Effects:** Options include:
  - Motion blur, depth of field (basic)
  - Alpha shadow (basic)
  - Cartoon style rendering (2pts)
  - Chromatic dispersion (2pts)
- **Lighting:** Options include:
  - Point light and area light (basic)
  - Environment lighting with HDR, such as skybox (2pts)
- **Anti-aliasing:** Implement an anti-aliasing algorithm (basic)
- **Simulation-based content creation:** Up to 2pts

## 5.2 Topic 2: Interactive Game

In this section, you are required to design an interactive game. The type and content of the game are flexible, but you must avoid using highly polished, complete game engines like Unity or UE5. A good starting point would be using Three.js. (If the engine you choose results in significantly less code than Three.js, you should be concerned about whether the workload meets the project's expectations). Additionally, the use of any engines that involve visual programming is strictly prohibited.

The following list includes the required basic features and optional features for your game:

- **Graphics Assets:**
  - Scene layout: reasonable object geometry, textures, and materials (basic)
  - Environment lighting (2pts)
  - Synchronized audio (2pts)
- **Animation:**
  - Kinematics:
    - Include animations, whether simple keyframe animations or more complex ones (basic)
    - Articulated objects: such as animals or plants, with rigging and skinning (3pts)
  - Dynamics (considering object interactions):
    - Fluid simulation: e.g., water or wind (3pts)
    - Soft/deformable objects (3pts)
    - Collision handling (3pts)
- **Interactive Control:** Ensure natural, intuitive, and smooth motions with controller inputs:
  - Control of the main game character (basic)
  - Camera motion control in either third-person or first-person view (basic, not required if developing a 2D game where the character stays at the center)
  - Implement suitable camera shakes during collisions or movement to enhance immersion (3pts)
- **UI Design:**
  - Proper game start and end interfaces (basic)
  - Additional auxiliary interfaces (3pts)
  - User-friendly layout with visually appealing design (2pts)
- **Advanced Rendering:**
  - Special visual effects such as motion blur, depth of field, etc. (3pts)
  - Path tracing rendering with PBR materials (3pts)
- **Other Advanced Game Features:**
  - Multi-player mode (3pts)
  - Support for various controllers, such as not only for mouse and keyboard, also for real controller (3pts)
  - Customizable character appearance (3pts)
  - Increasing difficulty levels throughout the game (3pts)
  - Save and load system, allowing players to save and resume their progress (3pts)
  - Online multiplayer system (4pts)
- **Completion:**
  - A fully functional, playable game (basic)
  - User manual or instructions (basic)
  - Easy installation or online access (3pts)

## 5.3 Topic 3: Physical Simulation

In this project, you are required to simulate physical interactions between objects and render the results using an industrial renderer. You must implement the core simulation algorithms yourself. General-purpose numerical libraries and programming frameworks, such as CUDA, NVIDIA Warp, and Taichi, are allowed, but ready-made physics engines and prebuilt simulation solvers are not. The functionalities are as follows:

- **Basic types**: Choose at least one (basic). Options include:
  - Rigid Body
  - Deformable Solid
  - Fluid
  - Cloth / Thin Shell
- **Physical interaction** (13 pts max):
  - Implement physical interactions between objects of the same or different types, including articulation, collision, self-collision, and coupling.
  - Full credit for an interaction type requires a meaningful two-way interaction.

| | Rigid | Deformable Solid | Fluid | Cloth / Thin Shell |
|---|---|---|---|---|
| Rigid | up to 5 pts | up to 6 pts | up to 7 pts | up to 6 pts |
| Deformable Solid | up to 6 pts | up to 5 pts | up to 7 pts | up to 6 pts |
| Fluid | up to 7 pts | up to 7 pts | up to 5 pts | up to 8 pts |
| Cloth / Thin Shell | up to 6 pts | up to 6 pts | up to 8 pts | up to 5 pts |

- **Geometry**
  - Trivial geometry (basic)
  - Complex (mesh-based or equation-based) geometry (up to 3 pts)
- **Acceleration**
  - Algorithmic acceleration (up to 5 pts). For example:
    - Acceleration structures for collision detection
    - Advanced numerical solvers
  - Parallel acceleration (up to 3 pts)
    - CPU multithreading, GPU acceleration, or a combination of both.
    - Credit depends on your implementation effort and measured speedup. Simply enabling a GPU backend, such as `ti.init(arch=ti.gpu)`, does not earn credit by itself.
    - CUDA and NVIDIA Warp are recommended for GPU acceleration.
- **Control**
  - Customized scene configuration (up to 5 pts)
  - Interactive scene. Options include:
    - Predefined operations during the simulation (up to 3 pts)
    - Real-time interaction (up to 3 pts)
- **Rendering**: Use an industrial renderer to produce a polished video (basic).
  - Visual presentation matters even though this is not a rendering project. Results shown only as raw simulation primitives, such as unshaded particles or line segments, will receive limited rendering credit.
  - Both offline and real-time rendering are acceptable.
  - Rendering is required. Additional credit is available based on the quality of the rendered results (up to 6 pts).

### 5.4 Topic 4: ProjectX

If none of the project topics mentioned above align with your interests, you have the option to propose a new project idea. In order to do so, you must create a grading sheet outlining all the technical aspects planned for your project, similar to the other projects listed earlier. Then, you need to make an appointment with TA before October 18 to obtain TA's consent. **You will only be allowed to pursue your own project idea if it is approved by the TAs.** Otherwise, you will still be required to select a topic from the aforementioned list as your final project.